Werkproces uitwerking SD-Portfolio

Werkproces	K1W5: Doet verbetervoorstellen voor de software
Studentnaam	Yakub Bulut
Studentnummer	97118569
Klascode	SDA3
Datum oplevering	10/20/2026

Student verklaart hierbij dat de bewijsstukken in dit STARR-document

- Authentiek zijn
- Kwalitatief goed zijn
- Binnen een redelijke tijdsduur tot stand zijn gekomen

Bewijsstuk

Per bewijsstuk moet er gekeken worden naar hoe sterk deze is, hiervoor kijk je naar twee dingen:

1. De context – Waar en met wie heb je het gedaan?
2. Je zelfstandigheid– Hoe zelfstandig heb je gewerkt?

Voor zowel de context als de zelfstandigheid kan een score A (hoog), B (middelmatig) of C (laag) gegeven worden. Een bewijsstuk met de score ‘A - A’ (goede context én zelfstandig gewerkt) heeft een hogere waarde dan een bewijsstuk met de score ‘B - C’.

Maak voor het bepalen van de complexiteit gebruik van de "Context check" en de "Zelfstandigheid check”.

Korte beschrijving	Context	Zelfstandigheid
In deze bewijsstuk laat ik zien hoe ik een verbetervoorstel heb gekregen van een vriend van mij die de functionaliteiten gemak op onze website ging testen en waar we feedback voor kregen	alleen Context check Uitkomst (B)	Context check Uitkomst (C)

STARR-reflectie

Situatie	Ik heb dit bewijsstuk op school gemaakt tijdens het werken aan mijn examenproject. De opdracht was om een verbetervoorstel te bedenken en uit te voeren. Tijdens het ontwikkelen van mijn reparatieaanvraagsysteem kreeg ik feedback van een vriend. Hij stelde voor om klanten een reparatieaanvraag te laten indienen zonder account. Op basis van deze feedback heb ik het systeem aangepast, zodat klanten zonder account een aanvraag kunnen doen en met een account hun reparatiestatus online kunnen volgen.
Taak	**Taakomschrijving 1:** Feedback verzamelen en beoordelen over het reparatieaanvraagsysteem. Hierbij heb ik gekeken naar het voorstel om een reparatieaanvraag zonder account mogelijk te maken. **Taakomschrijving 2:** Het verbetervoorstel uitwerken en bepalen welke aanpassingen nodig waren om het aanvraagformulier los te koppelen van het accountsysteem. **Taakomschrijving 3:** De aanpassing programmeren, zodat klanten zonder account een reparatieaanvraag kunnen indienen en ingelogde klanten hun reparatiestatus kunnen volgen. **Taakomschrijving 4:** De aangepaste functionaliteiten testen om te controleren of het indienen en volgen van reparatieaanvragen goed werkt.
Activiteiten	Tijdens het ontwikkelen van mijn reparatieaanvraagsysteem kreeg ik feedback van een vriend. Hij stelde voor om klanten een reparatieaanvraag te laten indienen zonder verplicht een account aan te maken. Ik heb eerst de routes in web.php aangepast, zodat het aanvraagformulier openbaar werd. Ook heb ik beveiliging toegevoegd tegen misbruik. Daarna heb ik het formulier aangepast, zodat gasten zelf hun naam en e-mailadres kunnen invullen. Vervolgens heb ik de controller aangepast om aanvragen zonder account te verwerken. Ook heb ik de database aangepast door user_id optioneel te maken en extra velden toe te voegen voor klantgegevens en een unieke statuscode. Vervolgens heb ik een persoonlijke statuslink toegevoegd, zodat gasten hun reparatie kunnen volgen via hun e-mail. Ook heb ik ervoor gezorgd dat gastaanvragen automatisch aan een account worden gekoppeld zodra het e-mailadres is bevestigd. Het eindresultaat is dat klanten zonder account een reparatie kunnen aanvragen, terwijl ingelogde klanten hun aanvragen via hun dashboard kunnen volgen. Hiermee heb ik de feedback verwerkt en het systeem gebruiksvriendelijker gemaakt.

3. Controller (app/Http/Controllers/RepairRequestController.php)

In `store()` kan `$user` nu `null` zijn (gast). Alleen gasten moeten naam en e-mail invullen. De mails gaan naar `customerEmail()` in plaats van `user->email`. Er is een nieuwe functie `status()` voor de openbare statuspagina.

```
--- a/app/Http/Controllers/RepairRequestController.php
+++ b/app/Http/Controllers/RepairRequestController.php
@@ -70,29 +70,51 @@ public function adminHistory()
         return view('admin.repairs.history', compact('repairs'));
     }

-    public function create()
+    // Formulier is openbaar: met account (ingelogd) of als gast.
+    public function create(Request $request)
     {
-        return view('customer.repairs.create');
+        // Gekozen apparaat op de Reparaties-pagina (?apparaat=laptop) alvast invullen
+        $device = collect(config('repairs.devices', []))
+            ->firstWhere('slug', $request->query('apparaat'));
+
+        return view('customer.repairs.create', [
+            'user' => auth()->user(),
+            'presetDevice' => $device['short'] ?? null,
+        ]);
```

```
    }


    public function store(Request $request)
    {
+        $user = $request->user();

-        $validated = $request->validate([
+        // Verborgen veld: echte bezoekers laten het leeg, spambots vullen het in.
+        if ($request->filled('website')) {
+            return redirect()->route('home');
+        }
+
+        // Gasten vullen zelf naam en e-mail in; bij een account nemen we die over.
+        $guestRules = $user ? [] : [
+            'name' => ['required', 'string', 'max:100'],
+            'email' => ['required', 'email', 'max:150'],
+        ];
```

1. Routes (routes/web.php)

De routes zijn uit de groep `auth` + `verified` gehaald en openbaar gemaakt. De oude situatie:

```
Route::middleware([
    'auth',
    'verified'
])->group(function () {
    // ...
    // Nieuwe reparatie aanvragen
    Route::get('/reparatie-aanvragen', [RepairRequestController::class, 'create'])-
>name('repairs.create');

    // Reparatie opslaan
    Route::post('/reparatie-aanvragen', [RepairRequestController::class, 'store'])-
>name('repairs.store');
```

Het verschil:

```
--- a/routes/web.php
+++ b/routes/web.php
```

6. Gastaanvragen koppelen aan een account

Maakt een gast later een account met hetzelfde e-mailadres, dan worden zijn aanvragen gekoppeld zodra hij inlogt of zijn e-mailadres bevestigt. Dat gebeurt alleen met een bevestigd e-mailadres, zodat niemand met andermans e-mailadres diens aanvragen kan inzien.

```
--- a/app/Models/User.php
+++ b/app/Models/User.php
@@ -41,4 +41,20 @@ public function repairRequests(): HasMany
     {
         return $this->hasMany(RepairRequest::class);
     }
+
+    /*
+    | Gastaanvragen met hetzelfde e-mailadres aan dit account koppelen.
+    | Alleen met een bevestigd e-mailadres, anders zou iemand met
+    | andermans e-mailadres diens aanvragen kunnen inzien.
+    */
+    public function claimGuestRepairRequests(): int
+    {
+        if (!$this->hasVerifiedEmail()) {
+            return 0;
+        }
+
+        return RepairRequest::whereNull('user_id')
+            ->whereRaw('LOWER(email) = ?', [mb_strtolower($this->email)])
+            ->update(['user_id' => $this->id]);
+    }
 }
\ No newline at end of file
--- a/app/Providers/AppServiceProvider.php
+++ b/app/Providers/AppServiceProvider.php
@@ -2,9 +2,12 @@

 namespace App\Providers;
```

`

Analyseren
Ik heb de feedback van mijn vriend bekeken en onderzocht hoe het reparatieaanvraagsysteem werkte. Hierbij heb ik de routes, het formulier en de controller gecontroleerd. Ik ontdekte dat klanten verplicht moesten inloggen en hun e-mailadres bevestigen. Daarom heb ik gekeken welke onderdelen aangepast moesten worden.

Verbeter-voorstellen
Op basis van de feedback heb ik voorgesteld om reparatieaanvragen zonder account mogelijk te maken. Daarnaast wilde ik dat klanten hun reparatiestatus konden blijven volgen. Dit heb ik opgelost door een persoonlijke statuslink toe te voegen en aanvragen automatisch aan een account te koppelen na e-mailverificatie.

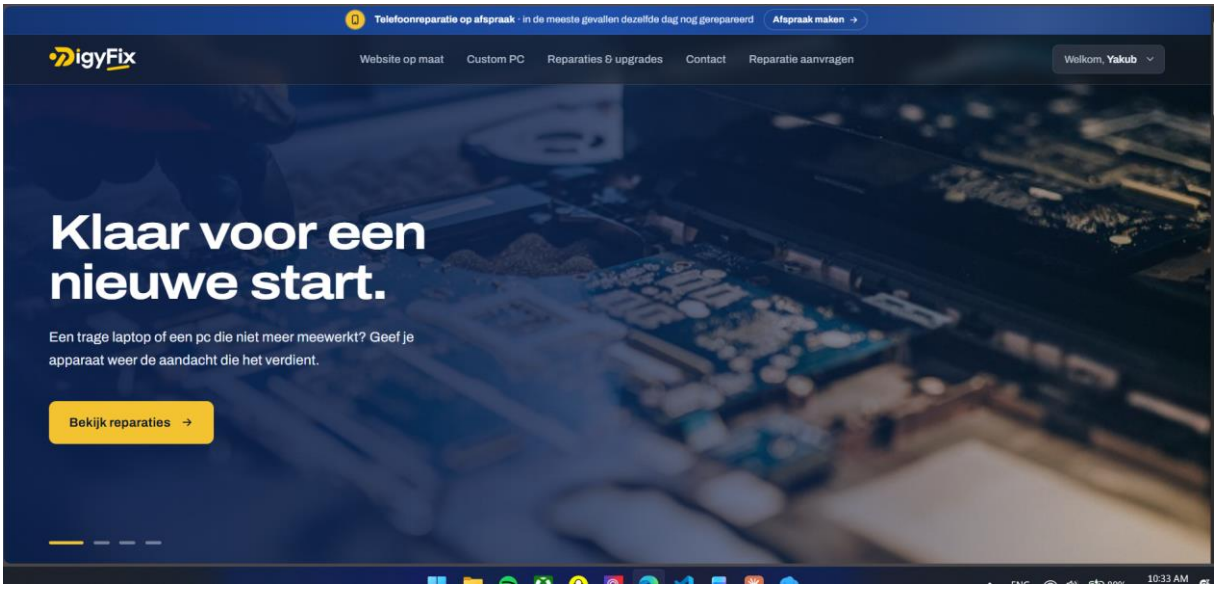
Planning
Ik heb bepaald welke onderdelen aangepast moesten worden. Eerst de routes en het formulier, daarna de controller en database. Als laatste heb ik de statuslink en accountkoppeling toegevoegd. De aanpassingen zijn op 16 oktober 2026 vastgelegd in Git.

Reflectie

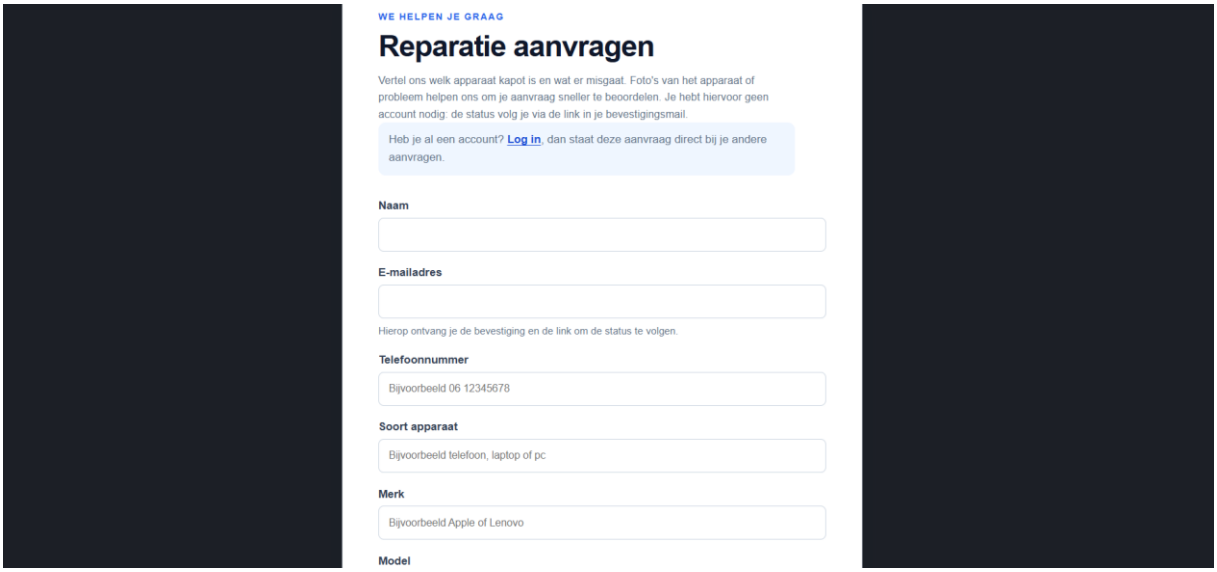
Feedback en feedforward
Ik kreeg feedback van een vriend dat het makkelijker zou zijn als klanten zonder account een reparatieaanvraag konden indienen. Deze feedback heb ik verwerkt. Voor de toekomst wil ik eerder feedback vragen aan gebruikers, zodat ik verbeterpunten sneller kan ontdekken.

	Kwaliteit van het eindproduct Ik ben tevreden met het eindresultaat. Klanten kunnen nu zonder account een reparatie aanvragen en hun status volgen via een persoonlijke link. Ook heb ik rekening gehouden met de beveiliging en gebruiksvriendelijkheid. **Tijd die nodig was** De aanpassingen zijn op 16 oktober 2026 vastgelegd in Git. De meeste werkzaamheden zaten in het aanpassen van de routes, controller, database en het toevoegen van de statuslink. **Hoe verliep het proces?** Het proces verliep goed. Ik heb eerst onderzocht wat er aangepast moest worden en daarna de verbeteringen uitgevoerd. Sommige onderdelen, zoals het koppelen van gastaanvragen aan accounts, waren wat ingewikkelder. **Samenwerking en overleg** De samenwerking verliep goed. Mijn vriend gaf mij het verbeterpunt en ik heb gekeken hoe ik dit technisch kon uitvoeren. Hierdoor heb ik geleerd om meer vanuit de gebruiker te denken. **Wat zou ik de volgende keer anders doen?** De volgende keer zou ik eerder feedback verzamelen en vooraf een duidelijkere planning maken. Ook zou ik gebruikers de nieuwe functionaliteit laten testen om te controleren of alles duidelijk en gebruiksvriendelijk is.

Bijlage 1.1



Bijlage 1.2



Bijlage 1.3

